

Path 2: Cyber Security 101 > Module 5: Networking > Room 4: Networking Secure Protocols

<https://tryhackme.com/room/networkingsecureprotocols>

Networking Secure Protocols:

Learn how TLS, SSH, and VPN can secure your network traffic.

>> Task 1: Introduction

In the **Networking Core Protocols** room, we learned about the protocols used to browse the web and access email, among others. These protocols work great; however, they cannot protect the confidentiality, integrity, or authenticity of the data transferred. In simpler terms, when we say that confidentiality is not protected, it means that someone watching the packets can read your password or credit card information when sent over HTTP. Similarly, they can access your private documents when sent via email. As for not protecting the integrity of the data, it means that an adversary can change the contents of the transferred data; in other words, if you authorise the payment of one hundred pounds, they can easily change it to another value, such as eight hundred pounds. Authenticity means ensuring we are talking with the correct server, not a fake one. Important online transactions are risky without ensuring confidentiality, integrity, and authenticity.

Transport Layer Security (TLS) is added to existing protocols to protect communication confidentiality, integrity, and authenticity. Consequently, HTTP, POP3, SMTP, and IMAP become HTTPS, POP3S, SMTPS, and IMAPS, where the appended "S" stands for Secure. We will examine these protocols and the benefits we reaped from TLS.

Similarly, it is deemed insecure to remotely access a system using the TELNET protocol; Secure Shell (SSH) was created to provide a secure way to access remote systems. Furthermore, SSH is an extensible protocol that offers added security features for other protocols.

♦ Learning Objectives:

Upon finishing this room, you will learn about:

- SSL/TLS

- How to secure existing plaintext protocols:

- HTTP

- SMTP

- POP3

- IMAP

- How SSH replaced the plaintext TELNET

- How VPN creates a secure network over an insecure one

>> Task 2: TLS

At one point, you would only need a packet-capturing tool to read all the chats, emails, and passwords of the users on your network. It was not uncommon for an attacker to set their network card in promiscuous mode, i.e., to capture all packets, including those not destined to it. They would later go through all the packet captures and obtain the login credentials of unsuspecting victims. There was nothing a user could do to prevent their login password from being sent in cleartext. Nowadays, it has become uncommon to come across a service that sends login credentials in cleartext.

In the early 1990s, Netscape Communications recognised the need for secure communication on the World Wide Web. They eventually developed SSL (Secure Sockets Layer) and released SSL 2.0 in **1995** as the first public version. In **1999**, the Internet Engineering Task Force (IETF) developed TLS (Transport Layer Security). Although very similar, TLS 1.0 was an upgrade to SSL 3.0 and offered various improved security measures. In **2018**, TLS had a significant overhaul of its protocol and TLS 1.3 was released. The purpose is not to remember the exact dates but to realise the amount of work and time put into developing the current version of TLS, i.e., TLS 1.3. Over more than two decades, there have been many things to learn from and improve with every version.

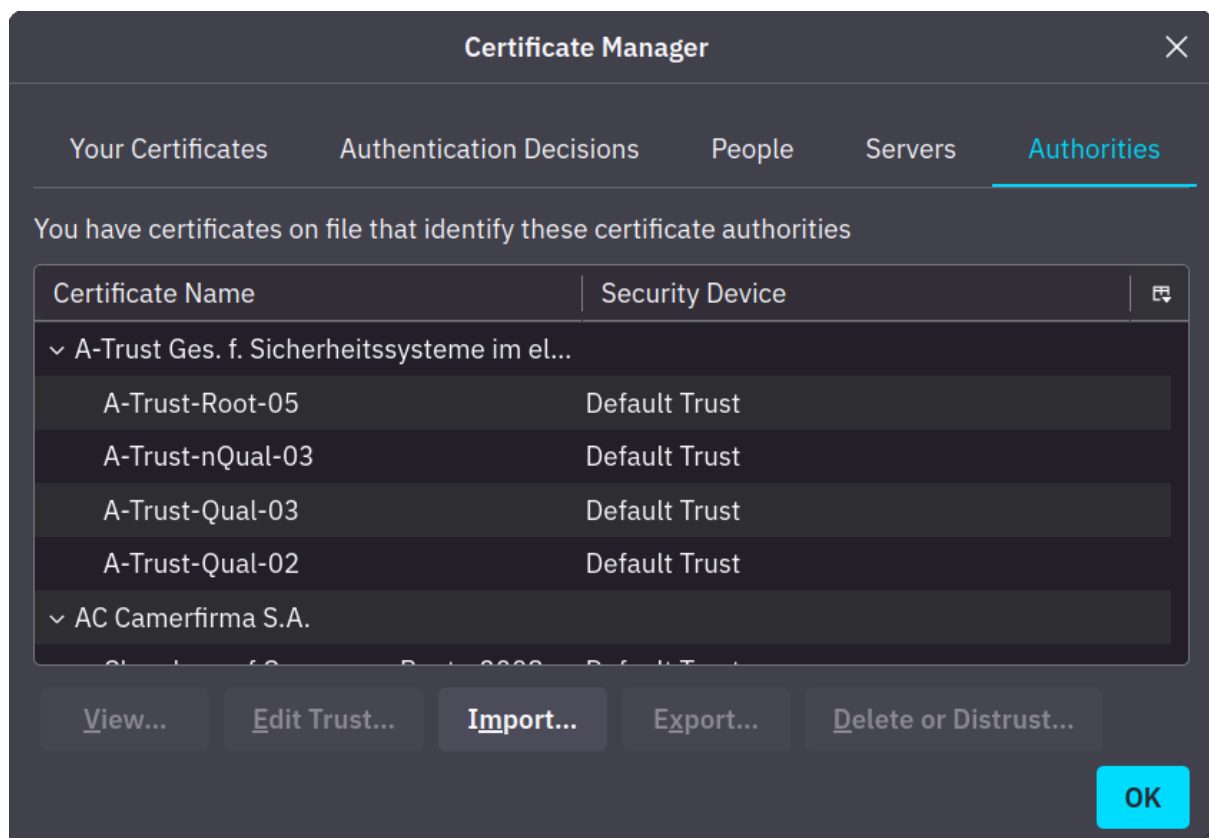
Like SSL, its predecessor, TLS is a cryptographic protocol operating at the OSI model's transport layer. It allows secure communication between a client and a server over an insecure network. By secure, we refer to confidentiality and integrity; TLS ensures that no one can read or modify the exchanged data. Please take a minute to think about what it would be like to do online shopping, online banking, or even online messaging and email without being able to guarantee the confidentiality and integrity of the network packets. Without TLS, we would be unable to use the Internet for many applications that are now part of our daily routine.

Nowadays, tens of protocols have received security upgrades with the simple addition of TLS. Examples include HTTP, DNS, MQTT, and SIP, which have become HTTPS, DoT (DNS over TLS), MQTTS, and SIPS, where the appended "S" stands for Secure due to the use of SSL/TLS. In the following tasks, we will visit HTTPS, SMTPS, POP3S, and IMAPS.

- ◆ Technical Background:

We will not discuss the TLS handshake; however, if you are curious, you can check the **Network Security Protocols** (<https://tryhackme.com/room/networksecurityprotocols>) room. We will give a general overview of how TLS is set up and used.

The first step for every server (or client) that needs to identify itself is to get a signed TLS certificate. Generally, the server administrator creates a Certificate Signing Request (CSR) and submits it to a Certificate Authority (CA); the CA verifies the CSR and issues a digital certificate. Once the (signed) certificate is received, it can be used to identify the server (or the client) to others, who can confirm the validity of the signature. For a host to confirm the validity of a signed certificate, the certificates of the signing authorities need to be installed on the host. In the non-digital world, this is similar to recognising the stamps of various authorities. The screenshot below shows the trusted authorities installed in a web browser.



Generally speaking, getting a certificate signed requires paying an annual fee. However, Let's Encrypt (<https://letsencrypt.org/>) allows you to get your certificate signed for free.

Finally, we should mention that some users opt to create a self-signed certificate. A self-signed certificate cannot prove the server's authenticity as no third party has confirmed it.

Q1) What is the protocol name that TLS upgraded and built upon?

Answer: SSL

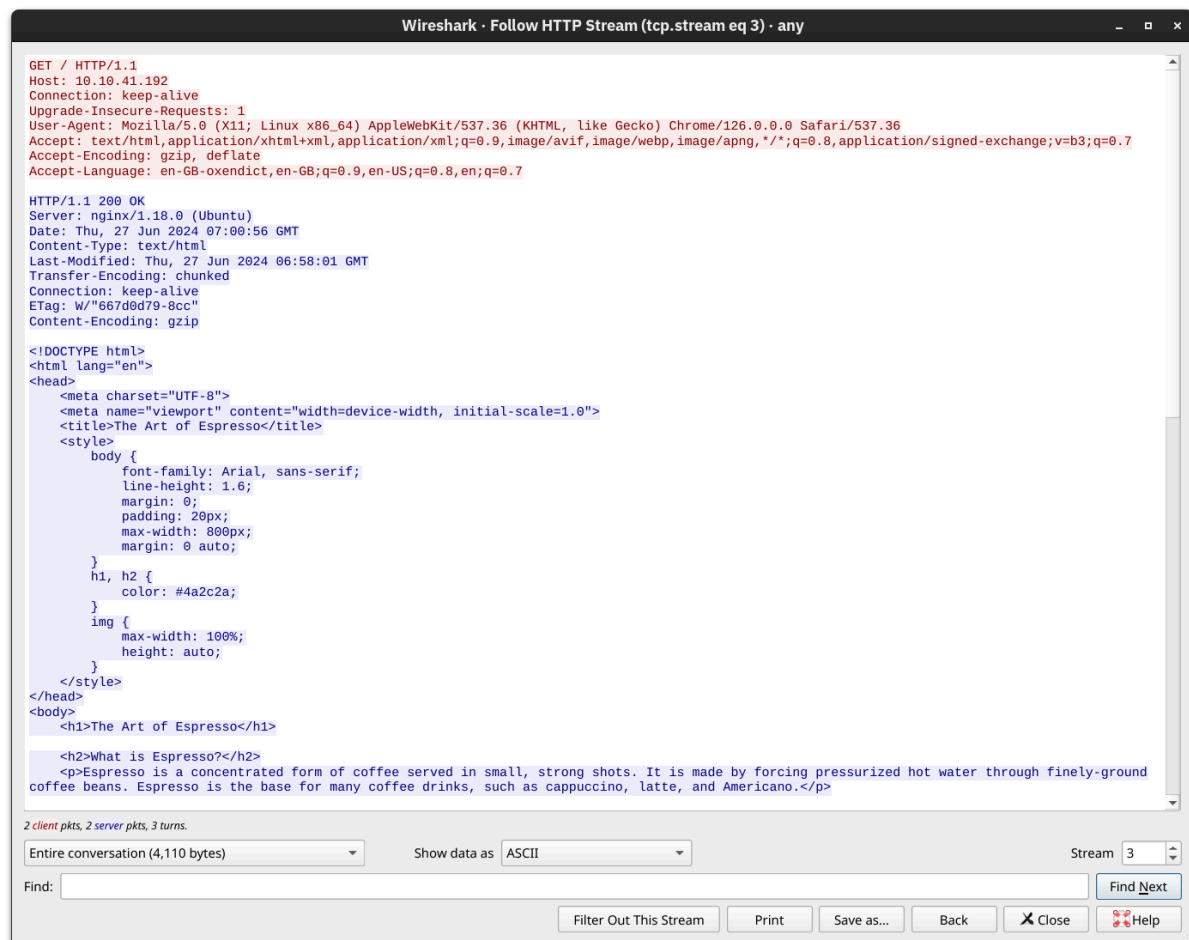
Q2) Which type of certificates should not be used to confirm the authenticity of a server?

Answer: self-signed certificate

>> Task 3: HTTPS

♦ HTTP:

As we studied in the **Networking Core Protocols** room, HTTP relies on TCP and uses port 80 by default. We also saw how all HTTP traffic was sent in cleartext for anyone to intercept and monitor. The screenshot below is from the previous room, and it gives a clear idea of how an adversary can easily read all the traffic exchanged between the client and the server.



Let's take a minute to review the most common steps before a web browser can request a page over HTTP. After resolving the domain name to an IP address, the client will carry out the following two steps:

Establish a TCP three-way handshake with the target server

Communicate using the HTTP protocol; for example, issue HTTP requests, such as

GET / HTTP/1.1

The two steps described above are shown in the window below. The three packets for the TCP handshake (marked with 1) precede the first HTTP packet with "GET" in it. The HTTP communication is marked with 2. The last three displayed packets are for TCP connection termination and are marked with 3.

The screenshot shows a Wireshark packet capture for 'HTTP-esspresso.pcapng'. The packet list table is as follows:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.11.81.126	10.10.41.192	TCP	76	52490 → 80 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PERM=1
2	0.073073828	10.10.41.192	10.11.81.126	TCP	76	80 → 52490 [SYN, ACK] Seq=0 Ack=1 Win=62643 Len=0 MSS=1460
3	0.073126006	10.11.81.126	10.10.41.192	TCP	68	52490 → 80 [ACK] Seq=1 Ack=1 Win=32120 Len=0 TSval=23161
4	0.073309310	10.11.81.126	10.10.41.192	HTTP	518	GET / HTTP/1.1
5	0.145362461	10.10.41.192	10.11.81.126	TCP	68	80 → 52490 [ACK] Seq=1 Ack=451 Win=62643 Len=0 TSval=37161
6	0.145416192	10.10.41.192	10.11.81.126	TCP	1344	80 → 52490 [ACK] Seq=1 Ack=451 Win=62643 Len=1276 TSval=37161
7	0.145450126	10.11.81.126	10.10.41.192	TCP	68	52490 → 80 [ACK] Seq=451 Ack=1277 Win=31872 Len=0 TSval=37161
8	0.145466797	10.10.41.192	10.11.81.126	HTTP	167	HTTP/1.1 200 OK (text/html)
9	0.145478078	10.11.81.126	10.10.41.192	TCP	68	52490 → 80 [ACK] Seq=451 Ack=1376 Win=31872 Len=0 TSval=37161
10	0.203973981	10.11.81.126	10.10.41.192	HTTP	459	GET /favicon.ico HTTP/1.1
11	0.276349438	10.10.41.192	10.11.81.126	HTTP	460	HTTP/1.1 404 Not Found (text/html)
12	0.276389436	10.11.81.126	10.10.41.192	TCP	68	52490 → 80 [ACK] Seq=842 Ack=1768 Win=31872 Len=0 TSval=37161
13	0.641397061	10.10.41.192	10.11.81.126	TCP	68	80 → 38892 [FIN, ACK] Seq=1 Ack=1 Win=482 Len=0 TSval=37161
14	0.641600272	10.11.81.126	10.10.41.192	TCP	68	38892 → 80 [FIN, ACK] Seq=1 Ack=2 Win=249 Len=0 TSval=23161
15	0.709830939	10.10.41.192	10.11.81.126	TCP	68	80 → 38892 [ACK] Seq=2 Ack=2 Win=482 Len=0 TSval=376613

The packet details pane for packet 4 shows the following HTTP request:

```

GET / HTTP/1.1\r\n
Host: 10.10.41.192\r\n
Connection: keep-alive\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp
Accept-Encoding: gzip, deflate\r\n
Accept-Language: en-GB-oxendict,en-GB;q=0.9,en-US;q=0.8,en;q=0.7\r\n
\r\n

```

◆ HTTP Over TLS:

HTTPS stands for Hypertext Transfer Protocol Secure. It is basically HTTP over TLS. Consequently, requesting a page over HTTPS will require the following three steps (after resolving the domain name):

1. Establish a TCP three-way handshake with the target server
2. Establish a TLS session
3. Communicate using the HTTP protocol; for example, issue HTTP requests, such as **GET / HTTP/1.1**

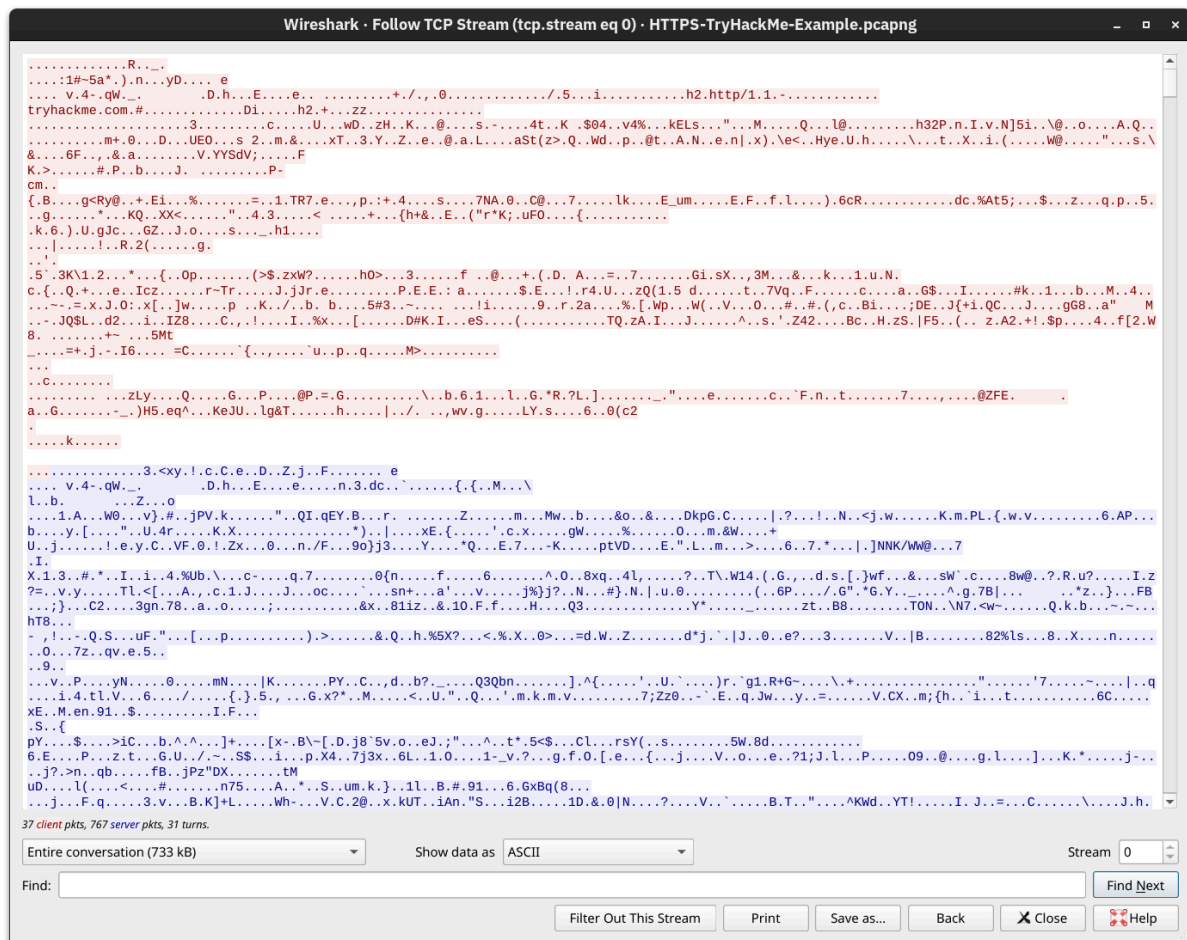
The screenshot below shows that a TCP session is established in the first three packets, marked with 1. Then, several packets are exchanged to negotiate the TLS protocol, marked with 2. 1 and 2 are where the **TLS negotiation and establishment** take place.

Finally, HTTP application data is exchanged, marked with **3**. Looking at the Wireshark screenshot, we see that it says “Application Data” because there is no way to know if it is indeed HTTP or some other protocol sent over port 443.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.124.148	172.67.27.10	TCP	74	55680 → 443 [SYN] Seq=0 Win=32120
2	0.057774975	172.67.27.10	192.168.124.148	TCP	74	443 → 55680 [SYN, ACK] Seq=0 Ack=1
3	0.057860075	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=1 Ack=1 Win=
4	0.058522508	192.168.124.148	172.67.27.10	TLSv1.3	1821	Client Hello (SNI=tryhackme.com)
5	0.114467329	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=1 Ack=1756 W
6	0.120054588	172.67.27.10	192.168.124.148	TLSv1.3	5129	Server Hello, Change Cipher Spec,
7	0.120085857	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=1756 Ack=566
8	0.123398977	192.168.124.148	172.67.27.10	TLSv1.3	130	Change Cipher Spec, Application Da
9	0.123579706	192.168.124.148	172.67.27.10	TLSv1.3	158	Application Data
10	0.123768581	192.168.124.148	172.67.27.10	TLSv1.3	1771	Application Data
11	0.176342271	172.67.27.10	192.168.124.148	TLSv1.3	578	Application Data, Application Data
12	0.176342521	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=5576 Ack=361
13	0.176380392	172.67.27.10	192.168.124.148	TLSv1.3	97	Application Data
14	0.176446477	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=3617 Ack=566
15	0.176542286	192.168.124.148	172.67.27.10	TLSv1.3	97	Application Data
16	0.272679122	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=5607 Ack=364
17	0.272679613	172.67.27.10	192.168.124.148	TLSv1.3	533	Application Data
18	0.272679813	172.67.27.10	192.168.124.148	TLSv1.3	99	Application Data
19	0.272679994	172.67.27.10	192.168.124.148	TLSv1.3	1202	Application Data
20	0.272680164	172.67.27.10	192.168.124.148	TLSv1.3	1457	Application Data

Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface eth0, id 0
Ethernet II, Src: 52:54:00:7c:d3:5b, Dst: 52:54:00:5d:c8:4d
Internet Protocol Version 4, Src: 192.168.124.148, Dst: 172.67.27.10
Transmission Control Protocol, Src Port: 55680, Dst Port: 443, Seq: 0, Len: 0

As expected, if one tries to follow the stream of packets and combine all their contents, they will only get gibberish, as shown in the screenshot below. The exchanged traffic is encrypted; the red is sent by the client, and the blue is sent by the server. There is no way to know the contents without acquiring the encryption key.



◆ Getting the Encryption Key

Adding TLS to HTTP leads to all the packets being encrypted. We can no longer see the contents of the exchanged packets unless we get access to the private key.

Although it is improbable that we will have access to the keys used for encryption in a TLS session, we repeated the above screenshots after providing the decryption key to Wireshark. The TCP and TLS handshakes don't change; the main difference starts conversation with the HTTP protocol marked 3. For instance, we can see when the client issues a **GET**.

HTTPS pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip.addr == 172.67.27.10

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.124.148	172.67.27.10	TCP	74	55680 → 443 [SYN] Seq=0 Win=32129
2	0.057774975	172.67.27.10	192.168.124.148	TCP	74	443 → 55680 [SYN, ACK] Seq=0 Ack=1
3	0.057860075	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=1 Ack=1 Win=
4	0.058522508	192.168.124.148	172.67.27.10	TLSv1.3	1821	Client Hello (SNI=tryhackme.com)
5	0.114467329	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=1 Ack=1756 W
6	0.120054588	172.67.27.10	192.168.124.148	TLSv1.3	5129	Server Hello, Change Cipher Spec
7	0.120085857	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=1756 Ack=536
8	0.123398977	192.168.124.148	172.67.27.10	TLSv1.3	130	Change Cipher Spec, Finished
9	0.123579706	192.168.124.148	172.67.27.10	HTTP2	158	Magic, SETTINGS[0], WINDOW_UPDATE[0]
10	0.123768581	192.168.124.148	172.67.27.10	HTTP2	1771	HEADERS[1]: GET /login
11	0.176342271	172.67.27.10	192.168.124.148	HTTP2	578	SETTINGS[0], WINDOW_UPDATE[0]
12	0.176342521	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=5576 Ack=331
13	0.176380392	172.67.27.10	192.168.124.148	HTTP2	97	SETTINGS[0]
14	0.176446477	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=3617 Ack=536
15	0.176542286	192.168.124.148	172.67.27.10	HTTP2	97	SETTINGS[0]
16	0.272679122	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=5607 Ack=334
17	0.272679613	172.67.27.10	192.168.124.148	HTTP2	533	HEADERS[1]: 200 OK
18	0.272679813	172.67.27.10	192.168.124.148	HTTP2	99	DATA[1]
19	0.272679994	172.67.27.10	192.168.124.148	HTTP2	1202	DATA[1]
20	0.272680164	172.67.27.10	192.168.124.148	TLSv1.3	1457	[TLS segment of a reassembled PDJ]

Frame 10: 1771 bytes on wire (14168 bits), 1771 bytes captured (14168 bits) on interface eth0, id 0

Ethernet II, Src: 52:54:00:7c:d3:5b, Dst: 52:54:00:5d:c8:4d

Internet Protocol Version 4, Src: 192.168.124.148, Dst: 172.67.27.10

Transmission Control Protocol, Src Port: 55680, Dst Port: 443, Seq: 1912, Ack: 5064, Len: 1705

Transport Layer Security

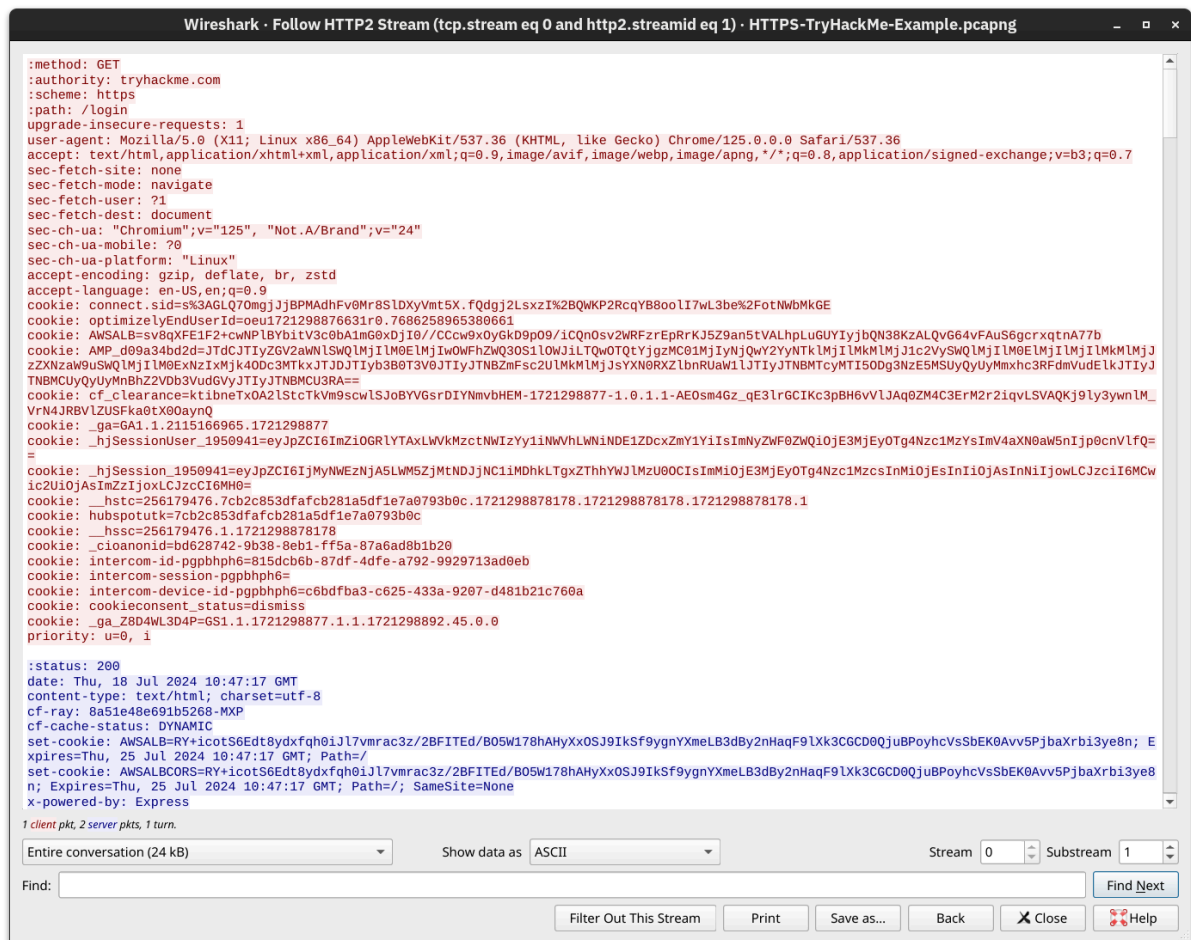
HyperText Transfer Protocol 2

- Stream: HEADERS, Stream ID: 1, Length 1674, GET /login
 - Length: 1674
 - Type: HEADERS (1)
 - Flags: 0x25, Priority, End Headers, End Stream
 - 0... = Reserved: 0x0
 - 0000 0000 0000 0000 0000 0000 0001 = Stream Identifier: 1
 - Pad Length: 01

HTTPS.pcapng

Packets: 1186 · Displayed: 1186 (100.0%) Profile: Default

If you want to see the data exchanged, now is your chance! It is still regular HTTP traffic hidden from prying eyes.



The key takeaway is that TLS offered security for HTTP without requiring any changes in the lower or higher layer protocols. In other words, TCP and IP were not modified, while HTTP was sent over TLS the way it would be sent over TCP.

Q1) How many packets did the TLS negotiation and establishment take in the Wireshark HTTPS screenshots above?

Answer: 8

Hint: You need to count the packets to establish a TCP connection and the packets to negotiate the TLS session.

Q2) What is the number of the packet that contain the **GET /login** when accessing the website over HTTPS?

Answer: 10

Hint: Check the decrypted HTTPS screenshot. The packet number is displayed in the leftmost column.

>> Task 4: SMTPS, POP3S, and IMAPS

Adding TLS to SMTP, POP3, and IMAP is no different than adding TLS to HTTP. Similar to how HTTP gets an appended S for Secure and becomes HTTPS, SMTP, POP3, and IMAP become SMTPS, POP3S, and IMAPS, respectively. Using these protocols over TLS is no different than using HTTP over TLS; therefore, almost all the points from the HTTPS discussion apply to these protocols.

The insecure versions use the default TCP port numbers shown in the table below:

Protocol	Default Port Number
HTTP	80
SMTP	25
POP3	110
IMAP	143

The secure versions, i.e., over TLS, use the following TCP port numbers by default:

Protocol	Default Port Number
HTTPS	443
SMTPS	465 and 587
POP3S	995
IMAPS	993

TLS can be added to many other protocols; the reasoning and advantages would be similar.

Q1) If you capture network traffic, in which of the following protocols can you extract login credentials: SMTPS, POP3S, or IMAP?

Answer: IMAP

>> Task 5: SSH

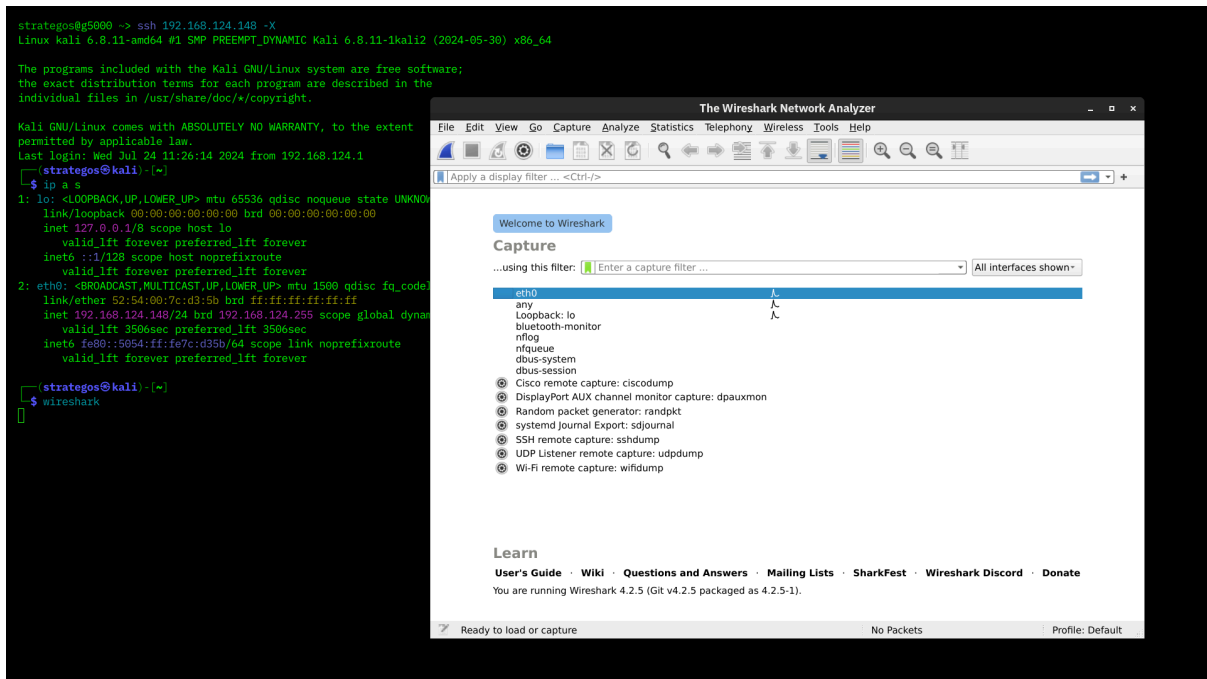
We have used the TELNET protocol in the **Networking Concepts** room. Although it is very convenient to log in and administer remote systems, it is risky when all the traffic is sent in cleartext. It is easy for anyone monitoring the network traffic to get hold of your login credentials once you use **telnet**. This problem necessitated a solution. Tatu Ylönen developed the Secure Shell (SSH) protocol and released SSH-1 in 1995 as freeware. (Interestingly, it was the same year that Netscape Communications released the SSL 2.0 protocol.) A more secure version, SSH-2, was defined in 1996. In **1999**, the OpenBSD developers released OpenSSH, an open-source implementation of SSH. Nowadays, when you use an SSH client, it is most likely based on OpenSSH libraries and source code.

OpenSSH offers several benefits. We will list a few key points:

- **Secure authentication**: Besides password-based authentication, SSH supports public key and two-factor authentication.
- **Confidentiality**: OpenSSH provides end-to-end encryption, protecting against eavesdropping. Furthermore, it notifies you of new server keys to protect against man-in-the-middle attacks.
- **Integrity**: In addition to protecting the confidentiality of the exchanged data, cryptography also protects the integrity of the traffic.
- **Tunneling**: SSH can create a secure “tunnel” to route other protocols through SSH. This setup leads to a VPN-like connection.
- **X11 Forwarding**: If you connect to a Unix-like system with a graphical user interface, SSH allows you to use the graphical application over the network.

You would issue the command **ssh username@hostname** to connect to an SSH server. If the username is the same as your logged-in username, you only need **ssh hostname**. Then, you will be asked for a password; however, if public-key authentication is used, you will be logged in immediately.

The screenshot below shows an example of running Wireshark on a remote Kali Linux system. The argument **-X** is required to support running graphical interfaces, for example, **ssh 192.168.124.148 -X**. (The local system needs to have a suitable graphical system installed.)



While the TELNET server listens on port 23, the SSH server listens on port 22.

Q1) What is the name of the open-source implementation of the SSH protocol?

Answer: OpenSSH

>> Task 6: SFTP and FTPS

SFTP stands for SSH File Transfer Protocol and allows secure file transfer. It is part of the SSH protocol suite and shares the same port number, 22. If enabled in the OpenSSH server configuration, you can connect using a command such as `sftp username@hostname`. Once logged in, you can issue commands such as `get filename` and `put filename` to download and upload files, respectively. Generally speaking, SFTP commands are Unix-like and can differ from FTP commands.

SFTP should not be confused with FTPS. You are right to think that FTPS stands for File Transfer Protocol Secure. How is FTPS secured? Yes, you are correct to estimate that it is secured using TLS, just like HTTPS. While FTP uses port 21, FTPS usually uses port 990. It requires certificate setup, and it can be tricky to allow over strict firewalls as it uses separate connections for control and data transfer.

Setting up an SFTP server is as easy as enabling an option within the OpenSSH server. Like HTTPS, SMTPS, POP3S, IMAPS, and other protocols that rely on TLS for security, FTPS requires a proper TLS certificate to run securely.

Exercise:

Securing Cleartext Protocols:

Matching Cleartext Protocol to Secure Protocol:

As we have learned in this module, every protocol uses a default port number, and the secure version uses a different default port number. In the following exercise, you need to match the cleartext protocol's default port number with the secure protocol's default port number.

For example, HTTP listens on TCP port 80 by default, and HTTPS listens on TCP port 443 by default. We should match 80 and 443.

Cleartext Protocol Port ↔ Secure Protocol Port: 80↔443, 23↔22, 143↔993, 25↔587, 110↔995, 21↔990.

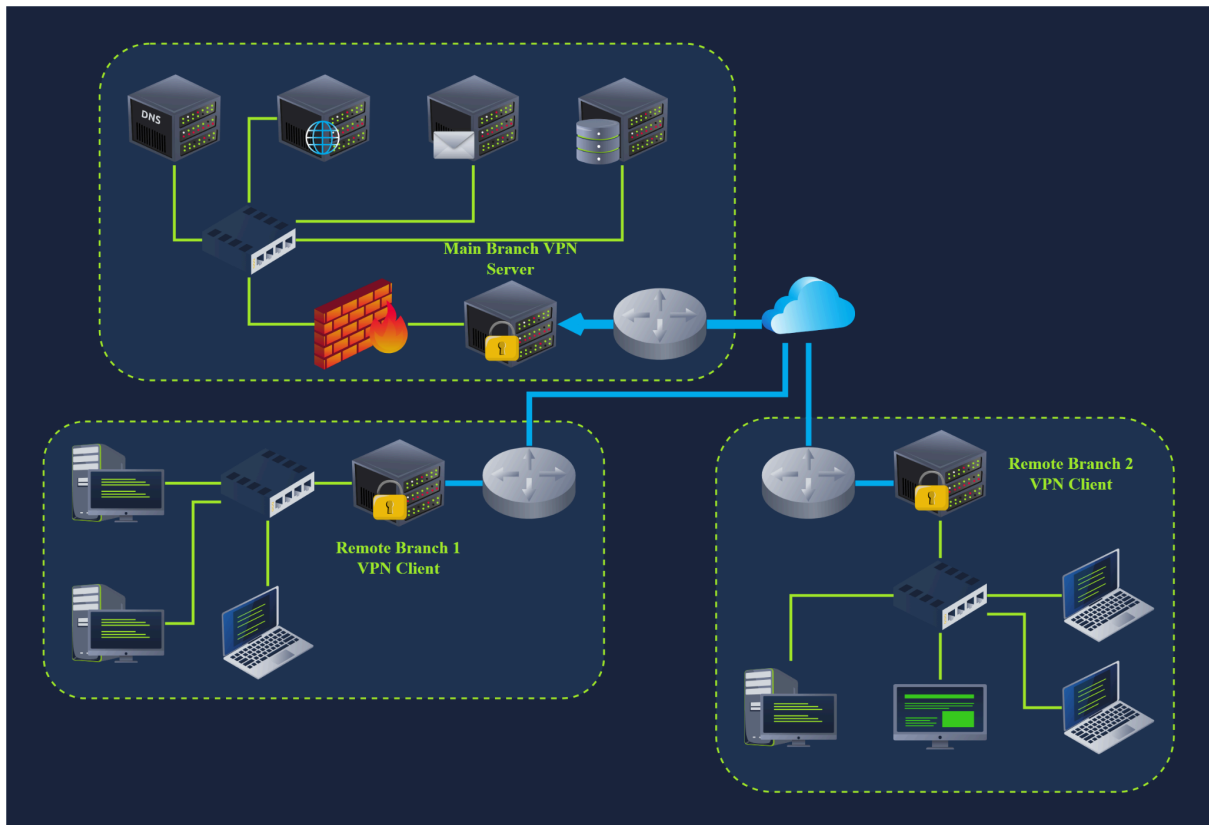
>> Task 7: VPN

Consider a company with offices in different geographical locations. Can this company connect all its offices and sites to the main branch so that any device can access the shared resources as if physically located in the main branch? The answer is yes; furthermore, the most economical solution would be setting up a virtual private network (VPN) using the Internet infrastructure. The focus here is on the V for Virtual in VPN.

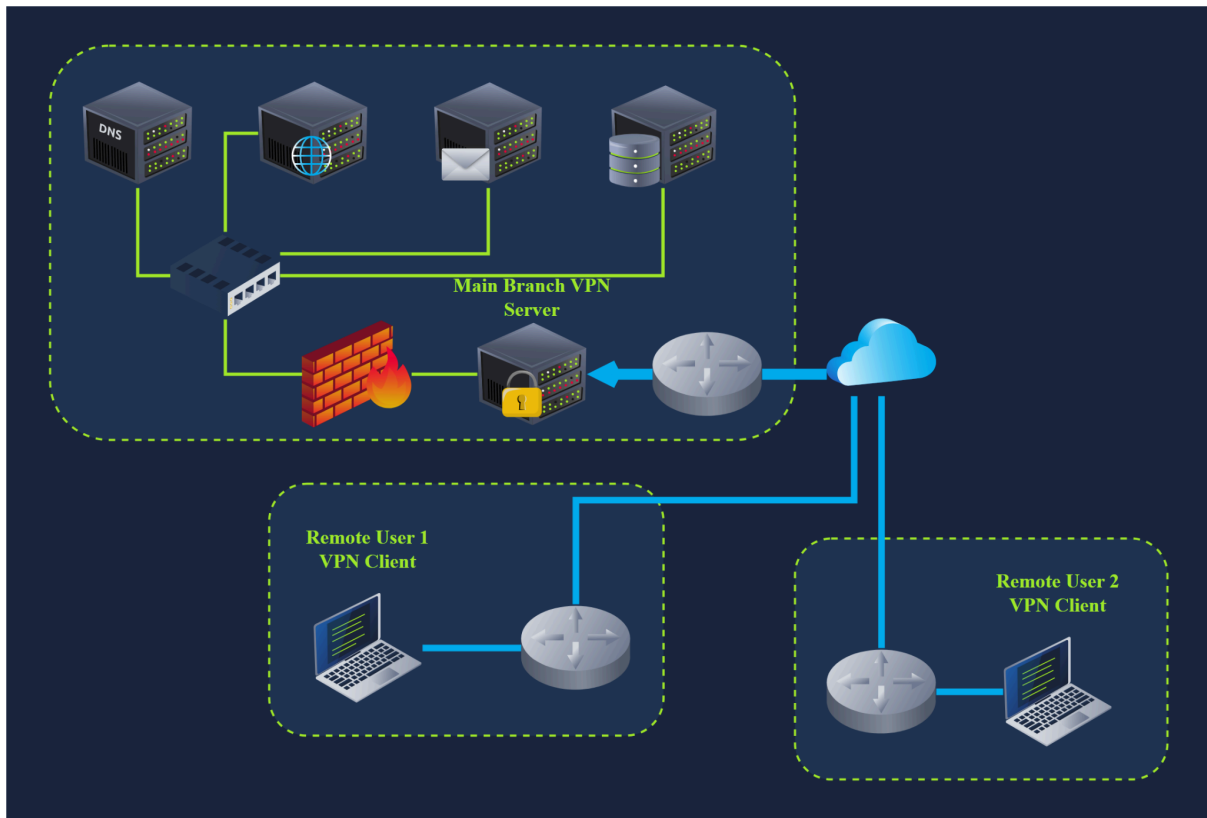
When the Internet was designed, the TCP/IP protocol suite focused on delivering packets. For example, if a router gets out of service, the routing protocols can adapt and pick a different route to send their packets. If a packet was not acknowledged, TCP has built-in mechanisms to detect this situation and resend. However, no mechanisms are in place to ensure that **all data** leaving or entering a computer is protected from disclosure and alteration. A popular solution was the setup of a VPN connection. The focus here is on the P for Private in VPN.

Almost all companies require “private” information exchange in their virtual network. So, a VPN provides a very convenient and relatively inexpensive solution. The main requirements are Internet connectivity and a VPN server and client.

The network diagram below shows an example of a company with two remote branches connecting to the main branch. A VPN client in the remote branches is expected to connect to the VPN server in the main branch. In this case, the VPN client will encrypt the traffic and pass it to the main branch via the established VPN tunnel (shown in blue). The VPN traffic is limited to the blue lines; the green lines would carry the decrypted VPN traffic.

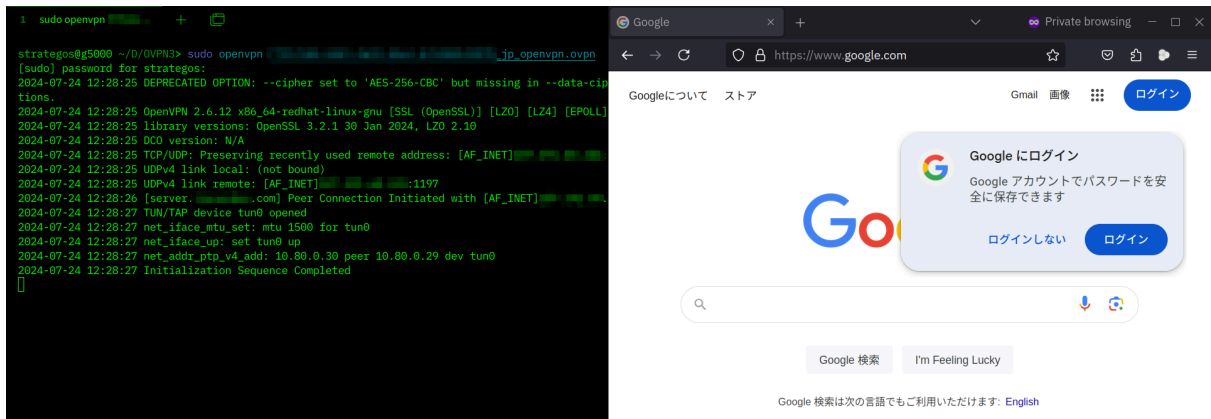


In the network diagram below, we see two remote users using VPN clients to connect to the VPN server in the main branch. In this case, the VPN client connects a single device.



Once a VPN tunnel is established, all our Internet traffic will usually be routed over the VPN connection, i.e. via the VPN tunnel. Consequently, when we try to access an Internet service or web application, they will not see our public IP address but the VPN server's. This is why some Internet users connect over VPN to circumvent geographical restrictions. Furthermore, the local ISP will only see encrypted traffic, which limits its ability to censor Internet access.

In other words, if a user connects to a VPN server in Japan, they will appear to the servers they access as if located in Japan. These servers will customise their experience accordingly, such as redirecting them to the Japanese version of the service. The screenshot below shows the Google Search page after connecting to a VPN server in Japan.



Finally, although in many scenarios, one would establish a VPN connection to route all the traffic over the VPN tunnel, some VPN connections don't do this. The VPN server may be configured to give you access to a private network but not to route your traffic. Furthermore, some VPN servers leak your actual IP address, although they are expected to route all your traffic over the VPN. Depending on why you are using a VPN connection, you might need to run a few more tests, such as a DNS leak test.

Finally, some countries consider using VPNs illegal and even punishable. Please check the local laws and regulations before using VPNs, especially while travelling.

Q1) What would you use to connect the various company sites so that users at a remote office can access resources located within the main branch?

Answer: VPN

>> Task 8: Closing Notes

The first approach is to use TLS, which provides a convenient way to secure many protocols, such as HTTP, SMTP, and POP3. Protocols secured with TLS usually get an S, for Secure, added to their names, such as HTTPS, SMTPS, and POP3S.

The second approach to secure network traffic is to use SSH. Although SSH is mainly used for remote access, it can also transfer files securely and establish secure tunnels. Creating an SSH tunnel is a solid choice if you want to pass the traffic of a plaintext protocol, such as VNC.

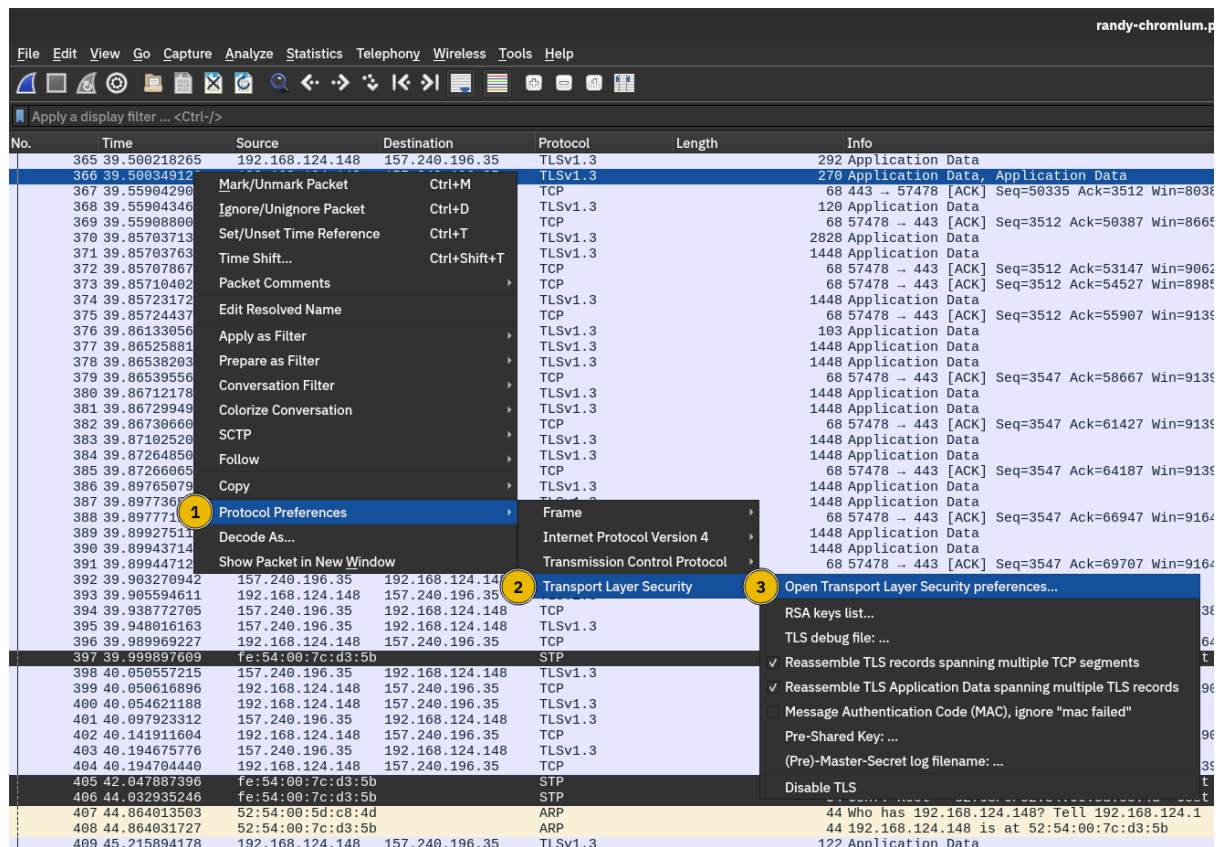
The last approach we covered to secure network traffic is using VPN connections. A VPN connection is usually the perfect option for connecting two company branches.

♦ Challenge:

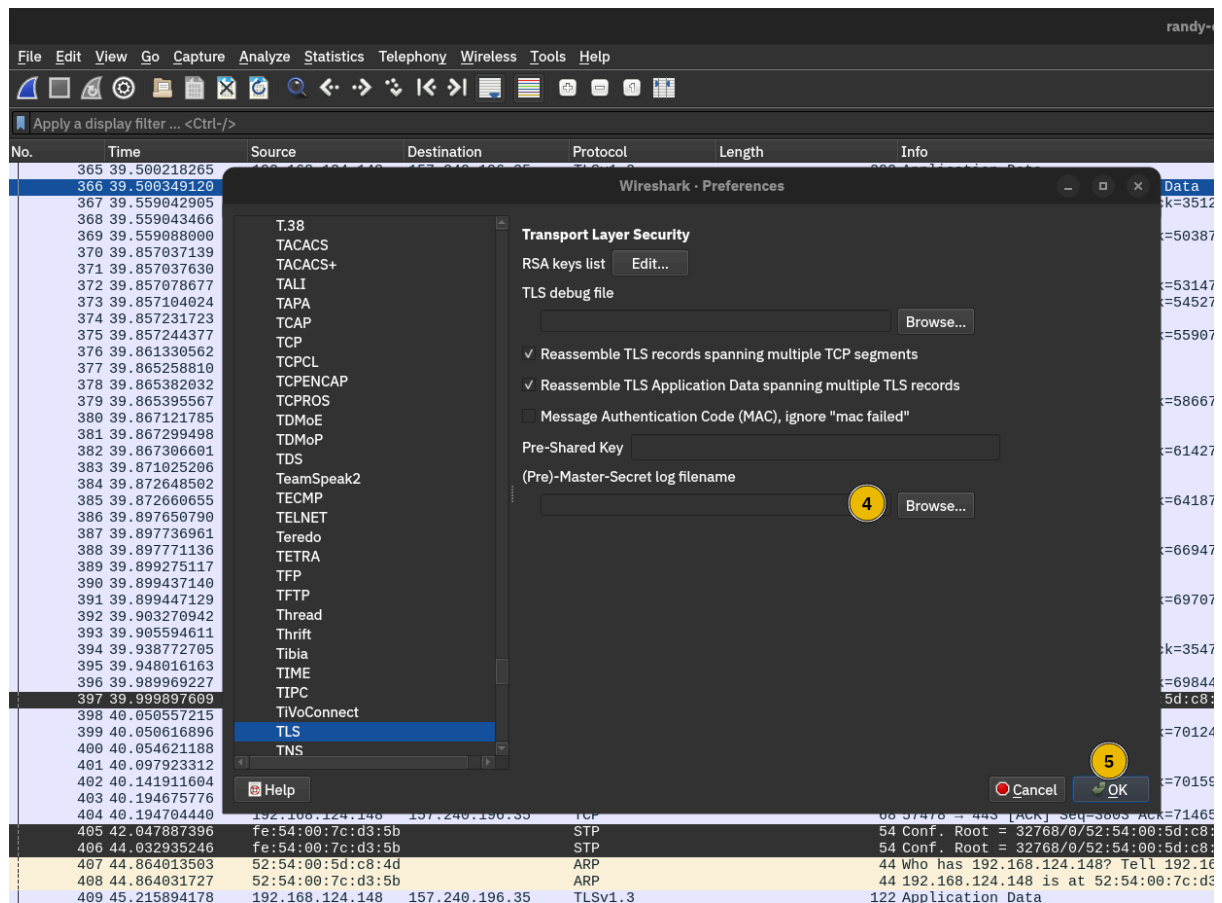
We have set the browser to log the session's TLS keys so we can take a closer look at the traffic using Wireshark. This logging was achieved by adding an extra option to the browser shortcut. Executing **chromium --ssl-key-log-file=~/.ssl-key.log** dumps the TLS keys to the **ssl-key.log** file.

The packet capture file is called **randy-chromium.pcapng** and is saved in the **Documents** folder. When you open the packet capture file in Wireshark, you can configure Wireshark to use the **ssl-key.log** file so that all the TLS traffic gets decrypted. You can see the five steps to achieve this in the two screenshots below.

First, after right-clicking any TLS packet, choose "Protocol Preferences." From the submenu, select "Transport Layer Security." Thirdly, click on "Open Transport Layer Security preferences."



Clicking “Open Transport Layer Security preferences” will show a dialog box. You must click the “Browse” button marked with four to locate the **ssl-key.log**. You can find it in the **Documents** directory. Finally, click OK, and Wireshark will show all the TLS decrypted. One of these packets contains login credentials.



Q1) One of the packets contains login credentials. What password did the user submit?

Answer: THM{B8WM6P}